```
================================================================================
TXT2PDF V1.8.0 - THE SPARTAN PDF GENERATOR
Zero-dependency TXT to PDF converter in pure C11
Now with zlib stream compression via libdeflate
================================================================================

"Turn plain text into structured PDFs without asking for permission.
Now the streams come out squeezed."

Language:    C11 (pure, no C++ runtime)
Compression: libdeflate (zlib / FlateDecode), statically embedded,
             enabled by default at level 12
Binary:      Static, stripped, musl libc, ~90-110 KB (architecture dependent)
Output:      PDF 1.4 with outlines, internal links, Base14 fonts,
             optional embedded Unicode TrueType fonts, images,
             zlib-compressed streams, and clickable URL links
Status:      v1.8.0 feature release

================================================================================
QUICK START
================================================================================

- Compile with the provided script (builds libdeflate and txt2pdf):
    ./compile.sh

- Run with defaults (Courier font, zlib compression level 12):
    ./txt2pdf readme.txt readme.pdf

- Disable compression (classic v1.7.2 output):
    ./txt2pdf --no-compress readme.txt readme.pdf

- Choose a compression level (1 fastest ... 12 smallest):
    ./txt2pdf --level 9 readme.txt readme.pdf

- Run with Helvetica:
    ./txt2pdf --sans readme.txt readme.pdf

- Run with Times-Roman:
    ./txt2pdf --serif readme.txt readme.pdf

- Run with an embedded Unicode TrueType font:
    ./txt2pdf --font NotoSansJP-Regular.ttf readme.txt readme.pdf

- Run with debug diagnostics:
    ./txt2pdf --debug readme.txt readme.pdf

- Version and help:
    ./txt2pdf --version
    ./txt2pdf --help

================================================================================
[TOC] TABLE OF CONTENTS
================================================================================

1.  what is txt2pdf?
2.  what is new in v1.8.0?
3.  fonts and typography
4.  stream compression (libdeflate)
5.  url links
6.  image support
7.  parser rules
```

8. table of contents rules
9. usage and debug mode
10. minimal working structure
11. pdf internals
12. project structure
13. building and static cookbook
14. known limitations
15. troubleshooting
16. design philosophy
17. license and credits

==========
1. WHAT IS TXT2PDF?
==========

txt2pdf is a zero-dependency command-line utility that converts plain text files into structured PDF 1.4 documents.

Unlike Pandoc, wkhtmltopdf, or Python-based converters that require hundreds of megabytes of runtime dependencies, txt2pdf is a small static binary.

It parses text, calculates page breaks, generates PDF objects, compresses the streams with zlib (libdeflate), and writes the final binary stream directly to disk using only the C standard library plus the statically embedded libdeflate compressor.

It is designed for environments where installing dependencies is impossible, unwanted, or where the network is down.

==========
2. WHAT IS NEW IN V1.8.0?
==========

Version 1.8.0 adds zlib stream compression via libdeflate, external URL support, and multi-architecture builds.

Main changes:

- Added zlib stream compression using libdeflate (/Filter /FlateDecode).
- Compression is enabled by default at level 12 (libdeflate range: 1-12).
- Added --level N to choose the compression level.
- Added --no-compress to generate uncompressed PDFs as in v1.7.2.
- --compress is still accepted for scripting (it is already the default).
- Compressed streams: page content streams, ToUnicode CMap and FontFile2.
- Each stream is checked: it is only compressed when the result is smaller.
- Images (JPEG/PNG) are never recompressed: they already carry their own DCTDecode or FlateDecode data.
- Added external URL support: http://, https://, ftp:// and www. links found in the text become clickable /URI link annotations, rendered in blue with underline.
- Added --version / -V and a cleaner --help / -h.
- Unknown options now produce an error instead of being ignored.
- Multi-architecture build script: compiles for amd64, arm64, and riscv64.
- Auto-downloads musl.cc toolchains if missing (no manual setup needed).
- Deterministic builds with SOURCE_DATE_EPOCH and ZERO_AR_DATE.
- Smaller binary: -Os plus garbage-collected sections (~90-110 KB).
- Each binary reports its platform (amd64 / arm64 / riscv64).
- A compression summary line is printed after each conversion.

Example compression impact (180-page technical manual):

- Uncompressed text streams: 824314 bytes
- Compressed, level 12:   261247 bytes
- Saved: 68.3 percent

All v1.7.2 features remain unchanged:

- CJK mode with --font FILE.ttf.
- TrueType subsetting.
- JPEG and PNG image passthrough.
- Outlines, internal links, UTF-8 and WinAnsi support.

==
3. FONTS AND TYPOGRAPHY
==

txt2pdf v1.8.0 supports two font pipelines.

A. Base14 fonts (classic mode):

--mono   Courier (default, classic txt2pdf layout)
--sans   Helvetica (clean, modern, proportional)
--serif   Times-Roman (classic documentation look)

Examples:

./txt2pdf --mono input.txt output.pdf
./txt2pdf --sans input.txt output.pdf
./txt2pdf --serif input.txt output.pdf

Important notes:

- Base14 fonts are not embedded into the PDF.
- PDF viewers normally provide built-in substitutes.
- Courier layout remains fixed-width and highly predictable.
- Helvetica and Times-Roman use built-in ASCII metrics for wrapping.
- Accents and non-ASCII characters remain limited to WinAnsiEncoding.

B. Embedded TrueType font (CJK mode):

--font FILE.ttf

Examples:

./txt2pdf --font NotoSansJP-Regular.ttf roadmap.txt roadmap.pdf
./txt2pdf --font NotoSansSC-Regular.ttf input.txt output.pdf
./txt2pdf --font NotoSansKR-Regular.ttf input.txt output.pdf

Important notes:

- The font must be an individual TrueType .ttf file.
- OpenType/CFF .otf files are not supported yet.
- TrueType collections .ttc are not supported yet.
- When --font is used, it overrides --sans, --serif and --mono.
- The selected font is used for ALL text (ASCII, Latin, CJK).
- Only the glyphs actually used in the document are embedded.
- The embedded FontFile2 stream is zlib-compressed by default.

==
4. STREAM COMPRESSION (LIBDEFLATE)
==

txt2pdf v1.8.0 compresses PDF streams with zlib using the libdeflate
library, which is compiled directly into the static binary.

What gets compressed:

- Page content streams (the actual text drawing operators).
- The ToUnicode CMap stream (CJK mode).
- The FontFile2 stream (embedded TrueType subset, CJK mode).

What is never recompressed:

- JPEG images (already DCTDecode).
- PNG images (already FlateDecode with PNG predictor).

Options:

```
(default)        Compression ON, zlib level 12.
--level N        Compression ON, level N (1 = fastest, 12 = smallest).
--compress       Explicit ON. Accepted for scripting; it is the default.
--no-compress    Compression OFF, classic v1.7.2 output.
```

Examples:

```
./txt2pdf input.txt output.pdf
./txt2pdf --level 6 input.txt output.pdf
./txt2pdf --no-compress input.txt output.pdf
```

Behavior notes:

- Each stream is checked: if compressing makes it bigger, it is stored
  uncompressed.
- Compressed streams are declared with /Filter /FlateDecode and the
  /Length entry always reflects the stored (compressed) size.
- Level 12 maximizes compression ratio but is slower. For huge documents
  try --level 6 for a good speed/size balance.
- At the end of each run a summary line is printed, for example:

  Compression: zlib level 12 (libdeflate) | streams 824314 ->
  261247 bytes (68.3% saved)

==========

5. URL LINKS

==========

External URLs in the text are automatically detected and converted to
clickable links.

Supported URL patterns:

- http://example.com
- ftp://files.example.org
- www.example.com (automatically prefixed with https://)

URLs are rendered in blue with underline (HTML-style) and are clickable
in any PDF viewer. Trailing punctuation (. , ; : ! ?) is automatically
stripped from the link.

Example:

http://example.com y www.example.org are working too.

All three URLs become clickable blue underlined links.

Limitations:

- URLs are only linked when they fit entirely inside one visual line.
- URL detection does not support line breaks within the URL.
- URLs inside TOC blocks are not scanned (TOC lines are reserved for
  internal links).

==
6. IMAGE SUPPORT
==

txt2pdf supports standalone block images.

Supported image tags:

[IMG]photo.jpg[/IMG]
[IMG]diagram.jpeg[/IMG]
[IMG]chart.png[/IMG]

The image line must be the only content on that line.

Good image line:

[IMG]chart.png[/IMG]

Bad image line:

See this chart: [IMG]chart.png[/IMG]

Supported formats:

- JPEG: Embedded using PDF native DCTDecode passthrough.
- PNG: Embedded using a minimalist passthrough for simple PNG files.

Supported PNG types:

- 8-bit grayscale
- 8-bit RGB
- 8-bit palette

Unsupported PNG types:

- alpha transparency
- interlaced PNGs
- 16-bit channels
- palette transparency

Image layout behavior:

- Images are treated as standalone content blocks.
- Aspect ratio is preserved.
- Images wider than the text area are scaled down.
- Images taller than the page are scaled down to fit.
- An image may force a page break if it does not fit.
- Image paths are relative to the current working directory.
- The PDF stores the original compressed image data untouched.
- Reducing source image size reduces final PDF size.

==

7. PARSER RULES

The parser creates PDF bookmarks only for lines that are real section headers.

A line is considered a section header only if all of these conditions are true:

- It starts with a digit, optionally followed by letters, then a dot.
- The title text after the dot contains only uppercase letters.
- The line is surrounded by separator lines made mostly of '=' characters.
- The line is not inside a contents block.

Valid section shape:

- separator line
- uppercase numbered title
- separator line

Rejected lines:

- numbered titles containing lowercase letters
- numbered titles used as examples inside normal paragraphs
- numbered titles not surrounded by separator lines
- numbered titles inside the contents block

This makes the parser much safer for manuals that contain examples.

The parser supports hierarchical IDs such as:

- 1
- 2.3
- 2.3.1
- 11.14
- 2.0a
- 13.1b
- 26.5a

8. TABLE OF CONTENTS RULES

Internal links are created only from contents blocks.

A contents block is detected when:

- a line contains the token "[TOC]"
- the next non-blank line is a separator line
- the block ends at the next separator line

Rules for entries:

- entries must be numbered lines
- lowercase entries are recommended
- entries are not converted into bookmarks
- entries are linked to the matching section by number or title
- uppercase entries inside the contents block are also ignored as bookmarks

This means the old duplicate-bookmark problem is largely eliminated by the parser itself.

9. USAGE AND DEBUG MODE

Basic usage:

```
./txt2pdf input.txt output.pdf
```

Font selection:

```
./txt2pdf --mono input.txt output.pdf
./txt2pdf --sans input.txt output.pdf
./txt2pdf --serif input.txt output.pdf
./txt2pdf --font FILE.ttf input.txt output.pdf
```

Compression control:

```
./txt2pdf input.txt output.pdf
./txt2pdf --level 6 input.txt output.pdf
./txt2pdf --compress input.txt output.pdf
./txt2pdf --no-compress input.txt output.pdf
```

Information:

```
./txt2pdf --help
./txt2pdf --version
```

Debug usage:

```
./txt2pdf --debug input.txt output.pdf
```

Combined usage:

```
./txt2pdf --debug --sans input.txt output.pdf
./txt2pdf --debug --level 9 input.txt output.pdf
./txt2pdf --debug --font NotoSansJP-Regular.ttf input.txt output.pdf
```

Debug mode prints diagnostics to stderr.

It reports:

- detected encoding
- selected font
- compression mode and level
- lines marked as part of a contents block
- detected section headers
- skipped uppercase numbered lines
- detected internal links
- detected images
- image placements
- CJK glyph count and subset size (in CJK mode)

Use debug mode when:

- a bookmark is missing
- too many bookmarks appear
- a contents entry does not link anywhere
- an image does not appear
- CJK characters render as boxes or ?
- you are adapting a new readme to the Spartan format

10. MINIMAL WORKING STRUCTURE

A minimal compatible document needs three things:

- a contents block starting with the [TOC] token
- lowercase or mixed-case contents entries
- uppercase section headers surrounded by separator lines

Recommended structure:

- write the contents title line containing [TOC]
- write a separator line
- write entries such as "1. introduction" and "2. conclusion"
- write a separator line
- for each section, write a separator line
- write the uppercase header, for example "1. INTRODUCTION"
- write another separator line
- write the section body

Important:

Do not wrap example headers with separator lines unless you want them to become real bookmarks.

11. PDF INTERNALS

txt2pdf generates a valid PDF 1.4 structure from scratch.

Catalog and Pages:

Calculates A4 dimensions, applies margins, wraps text, and places content using a line-based layout.

Fonts:

Generates a Base14 Type1 font object. The default is Courier. The --sans and --serif options switch to Helvetica or Times-Roman.

In CJK mode, it generates a Type0 font tree with a CIDFontType2 descendant, an embedded FontFile2 stream (subsetted TTF), and a ToUnicode CMap.

Streams and Compression:

When compression is enabled (the default), page content streams, the ToUnicode CMap and the FontFile2 stream are compressed with libdeflate in zlib format and declared with /Filter /FlateDecode. The /Length entry always reflects the stored compressed size. Streams that would grow when compressed are stored uncompressed instead. Image streams are never recompressed.

Images:

Loads image metadata, reserves vertical layout space, writes image XObject streams, and references them from page resources.

URL Links:

Scans text lines for http://, https://, ftp:// and www. patterns. Detected
URLs are rendered in blue with underline in the content stream and
converted to /URI link annotations.

Encoding and Smart Decoder:

Auto-detects UTF-8 vs Latin-1. If UTF-8 is detected, a smart decoder
translates supported multi-byte sequences to WinAnsiEncoding.

In CJK mode, UTF-8 codepoints are mapped directly to glyph IDs from the
embedded TrueType font, and the content stream uses hexadecimal notation
with Identity-H encoding.

Outlines:

Generates a linked list of PDF Outline objects pointing to the exact XYZ
coordinates of each uppercase header. In CJK mode, outline titles use
UTF-16BE hex encoding so they display CJK characters correctly.

Annotations:

Scans contents blocks and injects link annotations for each entry matching
a real section. URL links are injected as /URI annotations.

Cross-Reference Table:

Calculates byte offsets for every object and writes a compliant xref table
and trailer. Offsets are computed after compression, so they always point
to the exact start of each object.

==
12. PROJECT STRUCTURE
==

The txt2pdf codebase follows a single-file architecture plus one vendored
compression library.

Typical layout:

- txt2pdf.c
- compile.sh
- readme.txt
- README.md
- libdeflate/   (vendored zlib/deflate compressor sources)

The main C file contains:

- argument parsing
- encoding detection
- text parsing
- header detection
- contents detection
- image loading
- font selection (Base14 and TrueType)
- TrueType subsetting
- stream compression (libdeflate)
- URL detection
- page break calculation
- PDF object generation
- binary stream writing

The tool is intentionally small and auditable.

==========
13. BUILDING AND STATIC COOKBOOK
==========

Provided build script:

./compile.sh

What compile.sh does:

- Builds libdeflate as a static library.
- If libdeflate has a Makefile and make is available, it uses it.
- Otherwise it compiles the libdeflate/lib sources directly.
- CMake is NOT required.
- Verifies the library exports the needed symbols.
- Links txt2pdf against it as a fully static binary.
- Auto-downloads musl.cc toolchains if missing (for cross-compilation).
- Supports multi-architecture builds: amd64, arm64, riscv64.
- Uses deterministic build settings (SOURCE_DATE_EPOCH, ZERO_AR_DATE).

Build flags used to keep the binary small:

```
-Os -ffunction-sections -fdata-sections
-fno-unwind-tables -fno-asynchronous-unwind-tables
-static -Wl,--gc-sections -Wl,--build-id=none -s
```

Expected result:

```
wc -c txt2pdf
around 90-110 KB, static and stripped (architecture dependent)
```

Multi-architecture builds:

```
./compile.sh              (build for host architecture)
./compile.sh amd64        (build for x86_64)
./compile.sh arm64        (build for aarch64)
./compile.sh riscv64      (build for RISC-V 64-bit)
./compile.sh all          (build all three at once)
```

Manual build (once libdeflate/libdeflate.a exists):

```
musl-gcc -Os -Wall -Wextra -std=c11 -static -Ilibdeflate \
    -o txt2pdf txt2pdf.c libdeflate/libdeflate.a
strip txt2pdf
```

Why musl?

glibc static binaries are often much larger due to NSS and locale data.
musl provides a clean, minimal libc that results in a tiny, fully
self-contained executable.

Verify static build:

```
file txt2pdf
ldd txt2pdf
```

Expected:

- file reports statically linked
- ldd says it is not a dynamic executable

Optional extra squeeze (if UPX is installed):

```
upx --best --lzma txt2pdf
```

Force a libdeflate rebuild:

```
rm -rf libdeflate/build libdeflate/libdeflate.a
./compile.sh
```

14. KNOWN LIMITATIONS

General:

- No custom page sizes.
- No custom margins.
- No multi-column layout.
- No inline images.
- No image captions.
- No repeated header/footer images.
- No per-image width, height, DPI, or centering attributes yet.

Images:

- PNG support is limited to simple 8-bit non-interlaced images.
- PNG alpha transparency is not supported.
- Interlaced PNGs are not supported.

URLs:

- URLs are only linked when they fit entirely inside one visual line.
- URL detection does not support line breaks within the URL.
- URLs inside TOC blocks are not scanned.

Base14 mode:

- WinAnsiEncoding only.
- Characters outside WinAnsi are replaced with '?'.
- Base14 fonts are not embedded.

CJK mode:

- Requires an individual TrueType .ttf file.
- OpenType/CFF .otf files are not supported yet.
- TrueType collections .ttc are not supported yet.
- No automatic per-script font fallback.
- Scripts requiring shaping (Arabic, Indic, Thai) are not supported.

Compression:

- Only text-bearing streams are compressed (content, ToUnicode, FontFile2).
- Image streams are never recompressed.
- Level 12 is the slowest level; huge documents may prefer --level 6.
- libdeflate sources must be present in libdeflate/ at build time.

Parser:

- Section headers must be fenced by separator lines.
- Internal links are only generated from contents blocks.

15. TROUBLESHOOTING

Missing bookmarks:

- The header contains lowercase letters.
- The header is not surrounded by separator lines.
- The header is accidentally inside a contents block.

Duplicate bookmarks:

- Old documents may have uppercase contents entries.
- The updated parser ignores contents entries as sections.
- If duplicates persist, run with --debug and inspect skipped lines.

Missing links:

- The entry is not inside a detected contents block.
- The entry number does not match any section.
- The entry title does not match any section title.

Extra bookmarks:

- An example header is surrounded by separator lines.
- Remove the separators around examples or rewrite the example line.

URLs not detected:

- The URL contains line breaks (must fit on one visual line).
- The URL is inside a TOC block (TOC lines are not scanned for URLs).
- Trailing punctuation is stripped automatically.

Garbled characters in Base14 mode:

- Ensure your text editor saves the file in UTF-8.
- Run with --debug to verify the detected encoding.
- Characters outside WinAnsiEncoding cannot be rendered.
- Use --font FILE.ttf for full Unicode support.

CJK characters appear as boxes or '?' in CJK mode:

- The .ttf file does not contain the requested glyphs.
- Use a font with broader coverage (e.g. Noto Sans SC for Simplified
  Chinese, Noto Sans TC for Traditional Chinese, Noto Sans JP for
  Japanese, Noto Sans KR for Korean).
- Check that the file is a real .ttf, not .otf or .ttc.

Cannot load TTF font error:

- The file is not in the current directory. Use absolute path.
- The file is .otf (CFF) instead of .ttf (TrueType).
- The file is a .ttc collection (not supported yet).
- The file is corrupted or incomplete.

Image does not appear:

- The file path is wrong.

- The image line contains extra text beside the tag.
- The image format is not supported.
- The PNG uses alpha, interlacing, or 16-bit channels.

Image is too large:

- Per-image width or DPI attributes are not supported yet.
- Use a smaller source image or crop it.
- High-resolution images are scaled visually but still embedded.

PDF is not smaller with compression:

- The document is dominated by images; only text streams are compressed.
- Check the final "Compression:" summary line printed by txt2pdf.
- Verify you are not passing --no-compress.

libdeflate build problems:

- compile.sh does not use CMake; it is not required.
- If linking reports missing symbols, delete libdeflate/build and
  libdeflate/libdeflate.a and run compile.sh again.
- Make sure the libdeflate sources are complete (fresh clone or release
  tarball); the library needs all files under libdeflate/lib.

Font does not look as expected:

- Use --mono for the classic Courier layout.
- Use --sans for Helvetica.
- Use --serif for Times-Roman.
- Use --font FILE.ttf for Unicode CJK layout.
- Some viewers substitute Base14 fonts with similar fonts.

Debug output is the fastest way to diagnose the problem.

16. DESIGN PHILOSOPHY

Modern software development has forgotten how to build small tools.

We wrap simple text transformations in Electron apps, Node.js runtimes,
and Docker containers.

txt2pdf is a rejection of that bloat.

It proves that a fully functional, structured document generator can fit
in a tiny static binary, written in pure C11.

Version 1.8.0 bends the zero-dependency rule exactly once: the libdeflate
compressor is compiled directly into the binary. No shared libraries, no
runtime loading, no configuration files. One file, no excuses.

When the cloud goes down, and the package managers fail to resolve,
the Spartan tool remains.

"Code is neither created nor destroyed; it is distilled."

17. LICENSE AND CREDITS

Source code : GPLv3
Manual: GFDL v1.3
libdeflate: MIT license, Copyright 2016 Eric Biggers

txt2pdf was written as a demonstration of minimalist software design.

Inspired by the Unix philosophy: do one thing well.

Built with minimalism, on a potato, for potatoes.

https://github.com/johna124/txt2pdf
https://www.gnu.org/licenses/gpl-3.0.txt
https://www.gnu.org/licenses/fdl-1.3.txt

==========
END OF DOCUMENTATION
==========